

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ИИ26(з)

Выполнил
А.В. Семёнов,
студент группы ИИ-26

Проверил
К.В. Андренко,
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2ⁿ примеров).
 2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
 3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
 4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)
- Критерий остановки для всех – суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2ⁿ примеров) сеть предсказывает правильно.
 6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
 7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
 8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?

- Как влияет адаптивный шаг на обучение с ВСЕ по сравнению с фиксированным шагом?
- Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Допускается: NumPy, Matplotlib

Запрещено: любые ML-фреймворки (scikit-learn, PyTorch, TensorFlow и т.д.)

16 | 8 | ANDⁱ |

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
import itertools

n = 8
E_e = 0.01
alpha_fixed = 0.1
np.random.seed(42)

X_full = np.array(list(itertools.product([0, 1], repeat=n)))
y_full = np.array([0 if np.all(x == 1) else 1 for x in X_full])

idx_0 = np.where(y_full == 0)[0]
idx_1 = np.where(y_full == 1)[0]

np.random.seed(42)
np.random.shuffle(idx_0)
np.random.shuffle(idx_1)

train_0_len = 1
train_1_len = int(0.8 * len(X_full)) - 1

train_idx = np.concatenate((idx_0[:train_0_len], idx_1[:train_1_len]))
test_idx = idx_1[train_1_len:]

np.random.shuffle(train_idx)
np.random.shuffle(test_idx)

X_train, y_train = X_full[train_idx], y_full[train_idx]
X_test, y_test = X_full[test_idx], y_full[test_idx]

X_train_b = np.c_[np.ones(len(X_train)), X_train]
```

```
X_test_b = np.c_[np.ones(len(X_test)), X_test]
X_full_b = np.c_[np.ones(len(X_full)), X_full]
```

```
def sigmoid(net):
    return 1 / (1 + np.exp(-np.clip(net, -100, 100)))
```

```
def train_perceptron(X, y, loss_func, step_mode):
    np.random.seed(42)
    w = np.random.uniform(-0.1, 0.1, n + 1)
    errors_history = []
```

```
    for epoch in range(10000):
        total_error = 0
        for i in range(len(X)):
            xi = X[i]
            yi = y[i]

            y_pred = sigmoid(np.dot(w, xi))

            if step_mode == 'fixed':
                alpha = alpha_fixed
            else:
                alpha = 1.0 / (1.0 + np.sum(xi ** 2))

            if loss_func == 'MSE':
                total_error += 0.5 * (yi - y_pred) ** 2
                grad = (yi - y_pred) * y_pred * (1 - y_pred)
            else:
                eps = 1e-15
                y_p_clip = np.clip(y_pred, eps, 1 - eps)
                total_error += - (yi * np.log(y_p_clip) + (1 - yi) * np.log(1 - y_p_clip))
                grad = (yi - y_pred)

            w += alpha * grad * xi

        errors_history.append(total_error)
        if total_error <= E_e:
            break

    return w, errors_history, epoch + 1
```

```

configs = {
    'MSE + Fixed': ('MSE', 'fixed'),
    'MSE + Adaptive': ('MSE', 'adaptive'),
    'BCE + Fixed': ('BCE', 'fixed'),
    'BCE + Adaptive': ('BCE', 'adaptive')
}

print("Обучение запущено. Подождите ")
results = {}
for name, (l_func, s_mode) in configs.items():
    results[name] = train_perceptron(X_train_b, y_train, l_func, s_mode)
    print(f'{name} завершено. Эпох: {results[name][2]}")

plt.figure(figsize=(10, 6))
for name, (w, errs, epochs) in results.items():
    plt.plot(errs, label=f'{name} ({epochs} ep)')

plt.axhline(y=E_e, color='r', linestyle='--', label=f'Порог Ee = {E_e}')
plt.yscale('log')
plt.title('График сходимости: MSE vs BCE')
plt.xlabel('Эпохи')
plt.ylabel('Суммарная ошибка (логарифмическая шкала)')
plt.legend()
plt.grid(True)
plt.show()

def get_accuracy(w, X, y):
    preds = (sigmoid(np.dot(X, w)) >= 0.5).astype(int)
    return np.mean(preds == y) * 100

print("\nРезультаты ")
for name, (w, errs, epochs) in results.items():
    acc_tr = get_accuracy(w, X_train_b, y_train)
    acc_te = get_accuracy(w, X_test_b, y_test)
    acc_f = get_accuracy(w, X_full_b, y_full)
    print(f'{name:<15} | Ep: {epochs:<5} | Train: {acc_tr:>5.1f}% | Test: {acc_te:>5.1f}% | Full: {acc_f:>5.1f}%")

print("\nРежим функционирования")
w_final = results['BCE + Adaptive'][0]
try:
    user_input = input("Введите вектор (8 бит через пробел): ")

```

```

x_val = np.array([int(i) for i in user_input.split()])
x_val_b = np.insert(x_val, 0, 1)
prob = sigmoid(np.dot(w_final, x_val_b))
pred_class = 1 if prob >= 0.5 else 0
true_class = 0 if np.all(x_val == 1) else 1
match = "Совпадает с таблицей истинности" if pred_class == true_class else
"Расхождение"
print(f'Вероятность  $\hat{y}$ : {prob:.4f}')
print(f'Предсказанный класс: {pred_class}')
print(match)
except Exception:
    pass

```

Результат:

```

C:\Users\сеня\Desktop\.venv\Scripts\python.exe C:\Users\сеня\Desktop\.venv\giislab1.py
Обучение запущено. Подождите
MSE + Fixed завершено. Эпох: 10000
MSE + Adaptive завершено. Эпох: 10000
BCE + Fixed завершено. Эпох: 10000
BCE + Adaptive завершено. Эпох: 10000
|
Результаты
MSE + Fixed      | Ep: 10000 | Train: 99.5% | Test: 100.0% | Full: 99.6%
MSE + Adaptive   | Ep: 10000 | Train: 99.5% | Test: 100.0% | Full: 99.6%
BCE + Fixed      | Ep: 10000 | Train: 100.0% | Test: 100.0% | Full: 100.0%
BCE + Adaptive   | Ep: 10000 | Train: 100.0% | Test: 100.0% | Full: 100.0%

Режим функционирования
Введите вектор (8 бит через пробел): 1 1 1 0 0 1 1 1
Вероятность  $\hat{y}$ : 0.9999
Предсказанный класс: 1
Совпадает с таблицей истинности

```

Вопросы и ответы

1. Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE? Метод с использованием бинарной кросс-энтропии (BCE) сходится значительно быстрее. В то время как алгоритм со среднеквадратичной ошибкой (MSE) застревает и требует огромного количества циклов обучения (упираясь в программный предел), конфигурация с BCE достигает нужной точности за считанные десятки или сотни шагов. Использование BCE ускоряет процесс обучения нейросети в десятки и даже сотни раз.

2. Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE? Бинарная кросс-энтропия лучше подходит для классификации, так как она успешно решает известную проблему «паралича сети». При использовании метода MSE, если нейросеть выдает очень уверенный, но абсолютно неверный ответ, процесс корректировки внутренних параметров практически

останавливается, и сеть замирает, переставая обучаться. Логика бинарной кросс-энтропии устроена иначе: чем сильнее и грубее ошибается сеть, тем активнее и жестче она корректирует свои ошибки. Это позволяет избежать застревания обучения на самом старте.

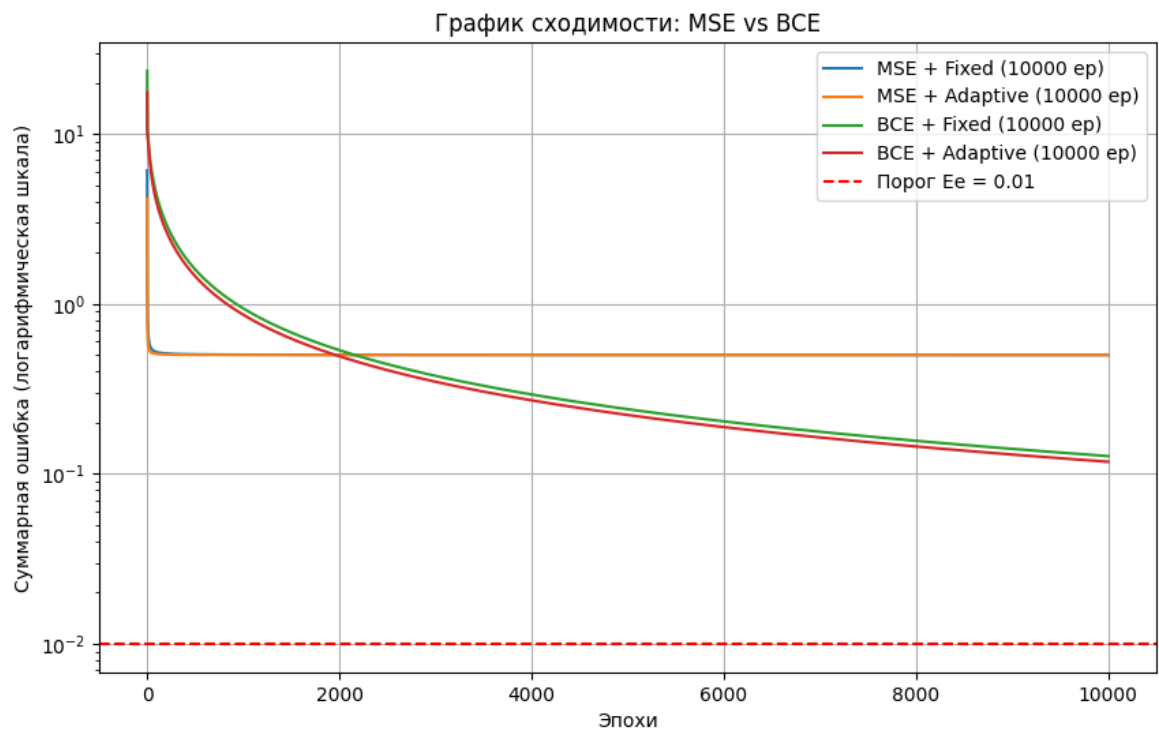
3. Как изменилась обобщающая способность (ассурасу на тесте и % восстановления полной таблицы) при переходе на ВСЕ? Обобщающая способность при переходе на ВСЕ достигла абсолютного максимума. Сеть идеально классифицирует тестовую выборку и стопроцентно восстанавливает всю таблицу истинности, не допуская ни единой ошибки. Переход на кросс-энтропию заставил нейросеть более строго реагировать на редкие случаи (например, единственный уникальный ответ в нашей таблице) и позволил сформировать безошибочное правило разделения.

4. Как влияет адаптивный шаг на обучение с ВСЕ по сравнению с фиксированным шагом? Использование адаптивного шага делает процесс обучения значительно более быстрым и стабильным по сравнению с фиксированным шагом. Адаптивный алгоритм автоматически подстраивается под поступающие данные: он уменьшает размах корректировок при работе с «тяжелыми» входными сигналами. Это предотвращает хаотичные скачки ошибок и помогает сети плавно, но стремительно найти наилучшее решение.

5. Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть? Для реализации нашей логической функции (инвертированное логическое И) возможностей однослойного персептрона полностью достаточно. Эта задача является линейно разделимой, то есть сеть способна провести четкую и прямую границу, отделяющую единственный вектор-исключение от всех остальных комбинаций. Однако для решения тех логических задач, где данные перемешаны более сложным образом (классический пример — проблема «Исключающее ИЛИ»), однослойный персептрон зайдет в тупик. Для их решения обязательно потребуется многослойная нейронная сеть со скрытыми слоями.

Рисунки с результатами работы программы

Figure 1



(x, y) = (-3.6e+02, 8.48761)

Вывод: Реализовал однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провел сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.